

Poison Pills are Too Toxic

Document #: P2602R1
Date: 2022-07-10
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 History](#)
- [4 Do we still need the poison pills?](#)
 - [4.1 ... except for iter_swap](#)
- [5 Proposal](#)
 - [5.1 Wording](#)
 - [5.2 Feature-Test Macro](#)
 - [5.3 Implementation Experience](#)
- [6 Acknowledgements](#)
- [7 References](#)

§ 1 Revision History

Since [[P2602R0](#)], added feature-test macro and wording for the comparison objects, which was previously missing.

§ 2 Introduction

Given the following declarations:

```
struct A {  
    friend auto begin(A const&) -> int const*;  
    friend auto end(A const&)   -> int const*;  
};  
  
struct B {  
    friend auto begin(B&) -> int*;  
    friend auto end(B&)  -> int*;  
};
```

B and `const A` satisfy `std::ranges::range`, but A does not. The goal of this paper is that both of these should count as ranges.

§ 3 History

During the design of Ranges, originally rvalues were prohibited entirely [[std2.429](#)]. Then, there was a desire ([[std2.547](#)])

to force range types to opt in to working with rvalues, thereby giving users a way to detect that, for a particular range type, iterator validity does not depend on the range's lifetime.

The problem is (as [[P0970R1](#)] demonstrates), when you have a construction like

```
struct Buffer { /* ... */ };

char* begin(Buffer&);
const char* begin(const Buffer&);
char* end(Buffer&);
const char* end(const Buffer&);
```

While `begin(Buffer{})` is valid, it's only valid because it binds to a const lvalue reference, and that doesn't actually offer any information as to whether the iterator would remain valid after the range is destroyed. That paper addressed this problem by introducing the concept of poison pill overloads. `std::ranges::begin(E)`, for rvalues `E`, will never consider `E.begin()` (since none of those are going to be ref-qualified) and will only consider overloads of `begin(E)` looked up in a context that includes:

```
// *Poison pill* overload:
template <class T>
void begin(T&&) = delete;
```

Here, `begin(Buffer{})` would prefer the poison pill, so we avoid that particular problem. `Buffer` is not a *borrowed* range (this is not the term P0970 used, but in an effort to limit confusion, I'll stick to the term we now have for this idea).

One reason this poison pill is necessary is that without it, `begin(std::vector<int>())` would happily find the `std::begin` overload that invokes member `begin()`, which is something we're trying to avoid.

As [[P1870R1](#)] pointed out, this is a very subtle distinction which makes it difficult to even properly opt into this concept. That paper proposed what is now the current opt-in model for borrowed ranges: specializing a variable template. `ranges::begin(E)` still only ever invokes `begin` (member or non-member) on an lvalue, but it decides whether it is allowed to upgrade an rvalue to an lvalue based on `enable_borrowed_range`.

§ 4 Do we still need the poison pills?

Given the existence of `enable_borrowed_range`, are the poison pills still useful? Well, we don't need to prevent lookup for `begin(Buffer{})` finding `::begin` or lookup for `begin(std::vector<int>())` from finding `std::begin`, since these types aren't borrowed ranges anyway.

For types that are borrowed ranges, there isn't any particular harm in finding `std::begin`, since that simply forwards to member `begin` (the customization point for `begin` is defined in such a way that

member begin takes priority to non-member begin, so had that one been valid we would have used it. Finding it doesn't add any value, but it also doesn't cause harm).

The poison pills were modified from one function template that takes a forwarding reference to two that take a const and non-const lvalue reference by [\[P2091R0\]](#), which contained this description:

While implementing this change, I realized that the forwarding-reference poison pills in the working draft are insufficiently poisonous. `void foo(auto&&)` is less-specialized than either `void foo(auto&)` or `void foo(const auto&)`, so a `void foo(auto&&)` poison pill fails to intercept/ambiguate calls to such overgeneric lvalue functions as intended. We should fix the poison pills by replacing them with two lvalue overloads. (I'm not certain the poison pills serve a useful design purpose anymore, and I'd like to remove them, but it's too late in the cycle for even so small a design change.)

Like the parenthetical says, it's not clear what we actually need the poison pills to poison anymore. It's clear what we needed to poison originally, but that motivation no longer exists. Unfortunately, their current formulation does cause harm.

First, consider [\[LWG3480\]](#). In that issue, `std::filesystem::directory_iterator` and `std::filesystem::recursive_directory_iterator` both opted into being ranges via non-member functions that looked like this:

```
directory_iterator begin(directory_iterator iter) noexcept;
directory_iterator end(const directory_iterator&) noexcept;

recursive_directory_iterator begin(recursive_directory_iterator iter)
    noexcept;
recursive_directory_iterator end(const recursive_directory_iterator&)
    noexcept;
```

The by-value overload was fine, but the reference-to-const one ends up being a worse match the `void end(auto&) = delete;` overload, so you end up in this state where `const directory_iterator` satisfies range but `directory_iterator` does not. This issue was resolved by simply making end for both of these iterator types take by value.

Second, consider [this example](#) from StackOverflow recently:

```
class Test {
    friend size_t size(/*const*/ Test&) {
        return 0;
    }
};

size_t f(Test t) {
    return std::ranges::size(t);
}
```

In order for this to compile, the `size` function must take a `Test&` (or a `Test`) and not a `const Test&`, even if no mutation is necessary. A member function `size() const` would've also sufficed, but this somewhat

defeats the purpose of allowing both member and non-member opt-in. Notice that here we're not even dealing with rvalue ranges - we're trying to invoke `size` with an lvalue, but it still won't work.

With the status quo, the poison pills prevent reasonable code from working and it's entirely unclear whether they prevent unreasonable code from working.

§ 4.1 ... except for `iter_swap`

I mentioned that there's no harm in lookup for `begin(e)` finding `std::begin(e)`, since that function template is constrained anyway to call `e.begin()`, which the `begin` customization point object tries to do anyway. A similar idea holds for all the other customization point objects.

Except for one: `iter_swap`. Having lookup for `iter_swap(i, j)` find `std::iter_swap(i, j)` is actually problematic because `std::iter_swap` has no constraints (see 27.7.3 [\[alg.swap\]](#)/6). As a result, `indirectly_swappable<I>` would end up holding for *any* input iterator `I` which has `std` as an associated namespace (that concept merely checks that `ranges::iter_swap(i1, i2)` is a valid expression, which, if `std::iter_swap` were found, would be the case). But not all such iterators are actually `indirectly_swappable` (e.g. `std::istreambuf_iterator<char>`), so that needs to continue to be true.

In order to preserve existing behavior, it is important to preserve the poison pill for `iter_swap`, which is basically:

```
void iter_swap(auto, auto) = delete;
```

This problem does not arise for `swap` however, as `std::swap` is properly constrained (22.2.2 [\[utility.swap\]](#)/1) and does the default thing anyway.

§ 5 Proposal

We can improve this situation by removing the poison aspect for the customization point objects that don't need them (i.e. all of them but `iter_swap`). We still need to, however, prevent regular unqualified lookup from happening and ensure that non-member lookup only happens in an argument-dependent lookup context. We need this both to prevent the CPO from finding itself (which would defeat the purpose of a non-member opt-in) and also to avoid looking up random nonsense in the global namespace. This matches what the language range-based for statement does, where 8.6.5 [\[stmt.ranged\]](#)/1.3.3 says:

- (1.3.3) otherwise, *begin-expr* and *end-expr* are `begin(range)` and `end(range)`, respectively, where `begin` and `end` undergo argument-dependent lookup ([\[basic.lookup.argdep\]](#)). [Note 1: Ordinary unqualified lookup ([\[basic.lookup.unqual\]](#)) is not performed. — *end note*]

The library implementation of this would be to effectively replace all the existing poison pills with nullary functions, as in:

```
- void begin(auto&) = delete;  
- void begin(const auto&) = delete;  
+ void begin() = delete;
```

§ 5.1 Wording

Change 16.3.3.3.6 [\[customization.point.object\]](#), since this note is describing the idea of a poison pill which is now going away:

- 7 ~~[Note 1: Many of the customization point objects in the library evaluate function call expressions with an unqualified name which results in a call to a program-defined function found by argument-dependent name lookup ([basic.lookup.argdep]). To preclude such an expression resulting in a call to unconstrained functions with the same name in namespace std, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace std. When the deleted overloads are viable, program-defined overloads need to be more specialized ([temp.func.order]) or more constrained ([temp.constr.order]) to be used by a customization point object. — end note]~~
- 7 When a customization point object is specified to use an expression with an unqualified name that undergoes argument-dependent lookup, ordinary unqualified lookup is not performed for that name.

Change 17.11.6 [\[cmp.alg\]](#) (just the sub-bullets that refer to doing non-member customization points):

- (1.2) — Otherwise, `strong_ordering(strong_order(E, F))` if it is a well-formed expression ~~with overload resolution performed in a context that does not include a declaration of `std::strong_order`~~ where `strong_order` undergoes argument dependent lookup. *[Note: Ordinary unqualified lookup is not performed. - end note]*
- ...
- (2.2) — Otherwise, `weak_ordering(weak_order(E, F))` if it is a well-formed expression ~~with overload resolution performed in a context that does not include a declaration of `std::weak_order`~~ where `weak_order` undergoes argument dependent lookup. *[Note: Ordinary unqualified lookup is not performed. - end note]*
- ...
- (3.2) — Otherwise, `partial_ordering(partial_order(E, F))` if it is a well-formed expression ~~with overload resolution performed in a context that does not include a declaration of `std::partial_order`~~ where `partial_order` undergoes argument dependent lookup. *[Note: Ordinary unqualified lookup is not performed. - end note]*

Change 25.3.3.1 [\[iterator.cust.move\]](#) [Drafting note: This isn't a behavior change, simply aligning the wording for all the customization point objects]:

- (1.1) — `iter_move(E)`, if `E` has class or enumeration type and `iter_move(E)` is a well-formed expression when treated as an unevaluated operand, ~~with overload resolution performed in a context that does not include a declaration of `ranges::iter_move` but does include the declaration `void iter_move();`~~ where `iter_move` undergoes argument dependent lookup. *[Note: Ordinary unqualified lookup is not performed. - end note]*

Add part of the old note into 25.3.3.2 [\[iterator.cust.swap\]](#):

- (4.1) — `(void)iter_swap(E1, E2)`, if either E1 or E2 has class or enumeration type and `iter_swap(E1, E2)` is a well-formed expression with overload resolution performed in a context that includes the declaration

```
template<class I1, class I2>
    void iter_swap(I1, I2) = delete;
```

and does not include a declaration of `ranges::iter_swap`. If the function selected by overload resolution does not exchange the values denoted by E1 and E2, the program is ill-formed, no diagnostic required. *[Note: This precludes calling unconstrained `std::iter_swap`. When the deleted overloads are viable, program-defined overloads need to be more specialized ([temp.func.order]) or more constrained ([temp.constr.order]) to be used. - end note]*

Change 26.3.2 [\[range.access.begin\]](#):

- (2.5) — Otherwise, if T is a class or enumeration type and `auto(begin(t))` is a valid expression whose type models `input_or_output_iterator` ~~with overload resolution performed in a context in which unqualified lookup for `begin` finds only the declarations~~

```
void begin(auto&) = delete;
void begin(const auto&) = delete;
```

where `begin` undergoes argument dependent lookup then `ranges::begin(E)` is expression-equivalent to that expression ~~`auto(begin(t))` with overload resolution performed in the above context.~~ *[Note: Ordinary unqualified lookup is not performed. - end note]*

Change 26.3.3 [\[range.access.end\]](#):

- (2.6) — Otherwise, if T is a class or enumeration type and `auto(end(t))` is a valid expression whose type models `sentinel_for<iterator_t<T>>` ~~with overload resolution performed in a context in which unqualified lookup for `end` finds only the declarations~~

```
void end(auto&) = delete;
void end(const auto&) = delete;
```

where `end` undergoes argument dependent lookup then `ranges::end(E)` is expression-equivalent to that expression ~~`auto(end(t))` with overload resolution performed in the above context.~~ *[Note: Ordinary unqualified lookup is not performed. - end note]*

Change 26.3.6 [\[range.access.rbegin\]](#):

- (2.4) — Otherwise, if T is a class or enumeration type and `auto(rbegin(t))` is a valid expression whose type models `input_or_output_iterator` ~~with overload resolution performed in a context in which unqualified lookup for `rbegin` finds only the declarations~~

```
void rbegin(auto&) = delete;
void rbegin(const auto&) = delete;
```

where `rbegin` undergoes argument dependent lookup then `ranges::rbegin(E)` is expression-equivalent to that expression ~~`auto(rbegin(t))`~~ with overload resolution performed in the above context. [Note: Ordinary unqualified lookup is not performed. - end note]

Change 26.3.7 [[range.access.rend](#)]:

- (2.4) — Otherwise, if τ is a class or enumeration type and `auto(rend(t))` is a valid expression whose type models `sentinel_for<decltype(ranges::rbegin(E))>` ~~with overload resolution performed in a context in which unqualified lookup for `rend` finds only the declarations~~

```
void rend(auto&) = delete;  
void rend(const auto&) = delete;
```

where `rend` undergoes argument dependent lookup then `ranges::rend(E)` is expression-equivalent to that expression ~~`auto(rend(t))`~~ with overload resolution performed in the above context. [Note: Ordinary unqualified lookup is not performed. - end note]

Change 26.3.10 [[range.prim.size](#)]:

- (2.4) — Otherwise, if τ is a class or enumeration type, `disable_sized_range<remove_cv_t<T>>` is `false` and `auto(size(t))` is a valid expression of integer-like type ~~with overload resolution performed in a context in which unqualified lookup for `size` finds only the declarations~~

```
void size(auto&) = delete;  
void size(const auto&) = delete;
```

where `size` undergoes argument dependent lookup then `ranges::size(E)` is expression-equivalent to that expression ~~`auto(size(t))`~~ with overload resolution performed in the above context. [Note: Ordinary unqualified lookup is not performed. - end note]

§ 5.2 Feature-Test Macro

Bump the value for `__cpp_lib_ranges` in 17.3.2 [[version.syn](#)]:

```
- #define __cpp_lib_ranges 202202L  
+ #define __cpp_lib_ranges 2022XXL  
// also in <algorithm>, <functional>, <iterator>, <memory>, <ranges>
```

This allows for only conditionally providing the additional overloads necessary to make mutable objects work.

§ 5.3 Implementation Experience

This has been implemented in both `libstdc++` and `MSVC`'s standard library.

§ 6 Acknowledgements

Thanks to Tim Song for help navigating everything, Casey Carter for the wording suggestions, and Jonathan Wakely for helping with libstdc++ testing. Thanks to Hui Xie for pointing out the comparison objects.

§ 7 References

[LWG3480] Barry Revzin. `directory_iterator` and `recursive_directory_iterator` are not C++20 ranges.

<https://wg21.link/lwg3480>

[P0970R1] Eric Niebler. 2018-05-06. Better, Safer Range Access Customization Points.

<https://wg21.link/p0970r1>

[P1870R1] Barry Revzin. 2019-11-25. `forwarding-range` is too subtle.

<https://wg21.link/p1870r1>

[P2091R0] Casey Carter. 2020-02-17. Issues with Range Access CPOs.

<https://wg21.link/p2091r0>

[P2602R0] Barry Revzin. 2022-06-17. Poison Pills are Too Toxic.

<https://wg21.link/p2602r0>

[`std`2.429] Casey Carter. 2018. Consider removing support for `rvalue` ranges from range access CPOs.

<https://github.com/ericniebler/std2/issues/429>

[`std`2.547] Eric Niebler. 2018. Redesign `begin/end` CPOs to eliminate deprecated behavior.

<https://github.com/ericniebler/std2/issues/547>